

Documentation Fonctionnelle et Technique - Jessica MLM (JTWc)

Plateforme MLM (Multi-Level Marketing) à arbre binaire — Symfony 6.4 / PHP 8.2

Document destiné aux développeurs reprenant le projet.

Chemin racine du projet Symfony : `C:\Users\tiome\Desktop\RACS
GLOBAL\jessica\symfony\`

1. Vue d'ensemble & stack

Jessica MLM (alias **JTWc** — Jessica The Winning Company) est une plateforme web de gestion d'un réseau de distributeurs en marketing de réseau (binaire). Elle gère :

- L'enregistrement et l'activation des distributeurs (avec parrain et placement dans un arbre binaire),
- Le catalogue produits (prix client / prix distributeur, valeurs SV/PV),
- Les commandes et leur paiement via **Dohone** (Mobile Money — Orange Money / MTN MoMo, devise XAF),
- Les abonnements aux paliers d'adhésion (**Membership**),
- Le calcul périodique (par cycle) des bonus : **bonus binaire**, **bonus de parrainage**, **bonus indirect**, **bonus générationnel**, **bonus spéciaux/promo**,
- La gestion des grades / rangs et leur maintenance,
- Les rapports (carry-over mensuel, récap activité réseau, états financiers).

Stack technique

Couche	Technologie
Langage	PHP 8.2 (<code>composer.json</code> ligne 5 : <code>"php": "^8.1"</code>)
Framework	Symfony 6.4 LTS (composants <code>6.4.*</code>)
ORM	Doctrine ORM 2.14+ avec extensions Gedmo (Tree Nested)
Base de données	MariaDB 10.4 (prod : container <code>jessica_db</code>)
Cache	Redis 7 en prod ; filesystem en dev (cf. <code>config/packages/dev/cache.yaml</code>)
Sérialisation	JMS Serializer Bundle
PDF	KnpsnappyBundle (wkhtmltopdf)
Images	LiipImagineBundle, VichUploaderBundle
Front-end	Webpack Encore, jQuery, FOSCKEditor, FOSJsRouting

Couche	Technologie
Pagination	KnpPaginatorBundle
Sécurité	security-bundle 6.4 + authenticator custom + Voter
Paieement	API HTTP Dohone (Mobile Money XAF) — App\Services\Payment\PayInWithDohone
Tests	PHPUnit 11.5, PHPStan 1.10
Déploiement	Docker Compose (3 conteneurs : jessica_web , jessica_db , jessica_redis), Nginx + Let's Encrypt sur jessica-mlm.duckdns.org . CI/CD GitHub Actions.

2. Architecture générale

Application Symfony classique (couches Controller -> Service/Handler -> Repository -> Entity), enrichie d'un pattern **Handler** (**src/Services/ModelHandlers/*Handler.php**) qui factorise le CRUD générique des entités d'administration (list/new/edit/delete + rendu Twig).

```

symfony/
■■■■ assets/                # JS/SCSS Webpack Encore (entry: app.js)
■■■■ bin/console
■■■■ config/
■   ■■■■ packages/          # cache, doctrine, security, mailer, monolog, vich_uploader, ...
■   ■■■■ routes/            # annotations.yaml -> auto-prefix /admin sur Back/WebController
■   ■■■■ services.yaml
■■■■ docker/                # Dockerfile(s), entrypoint Apache
■■■■ migrations/            # Doctrine Migrations (Version2019..2021)
■■■■ public/                # front controller + router.php
■■■■ src/
■   ■■■■ AbstractModel/     # interfaces communes (EntityInterface, EntityWithImageToUploadInterface)
■   ■■■■ Command/           # commandes CLI Symfony (1 commande : nettoyage paniers)
■   ■■■■ Controller/
■   ■   ■■■■ Back/WebController/ # backoffice admin (préfixe /admin)
■   ■   ■■■■ Front/WebController/ # placeholder (vide)
■   ■   ■■■■ SecurityController.php
■   ■■■■ DTO/, DataFixtures/, DependencyInjection/
■   ■■■■ Entity/             # 50+ entités Doctrine
■   ■■■■ EntityListener/, EventListener/, Event/
■   ■■■■ Exception/, Factory/
■   ■■■■ Form/               # 40+ FormType
■   ■■■■ Hydrators/
■   ■■■■ Manager/, Message/, MessageHandler/
■   ■■■■ Repository/         # repositories Doctrine
■   ■■■■ Security/           # LoginFormAuthenticator + UserChecker
■   ■■■■ Services/           # logique métier MLM
■   ■   ■■■■ ModelHandlers/   # CRUD generique réutilisable (UserHandler, ProductHandler, ...)
■   ■   ■■■■ Payment/         # PayInContext (Strategy) + PayInInterface + PayInWithDohone
■   ■■■■ Storage/, Validator/, Voter/
■   ■■■■ Kernel.php
■■■■ templates/
■   ■■■■ back/webcontroller/ # 30+ sous-répertoires (un par module métier)
■   ■■■■ form/, security/
■   ■■■■ base.html.twig
■   ■■■■ dashboard.html.twig / admin_dashboard.html.twig / user_dashboard.html.twig
■■■■ translations/, var/, vendor/
■■■■ docker-compose.yml, Dockerfile
■■■■ webpack.config.js

```

Conventions importantes

- Le préfixe `/admin` est ajouté **automatiquement** par `config/routes/annotations.yaml` (lignes 7-12) sur tous les controllers de `src/Controller/Back/WebController/`. **Ne jamais** écrire `/admin/...` dans une `#[Route(...)]` (provoquerait `/admin/admin/...`).
- Le pattern Handler : un controller délègue à `App\Services\ModelHandlers\<Entity>Handler` qui hérite de `ModelSingleEntityAbstract` ou `ModelCollectionEntityAbstract`. Ces classes savent : `list()`, `new()`, `edit()`, `delete()`, etc., en s'appuyant sur un Twig path et un FormType configurés.
- Les URLs publiques exposées au front via `FOSJsRoutingBundle` portent `options: ['expose' => true]`.

3. Modèle de données

3.1 Vue d'ensemble des entités (`src/Entity/`)

Plus de 50 entités. On peut les regrouper :

Utilisateurs / arbre

- `User` — distributeur ou admin. Cumule les rôles.
- `AddressUser` — adresse de livraison + téléphone Mobile Money utilisé pour les paiements.
- `UserBinaryCycle` — instantané du binaire d'un user pour un cycle donné.
- `UserMonthCarryOver` — report mensuel (carry-over) du SV non binarisé sur la jambe forte.
- `UserGrade` — historisation du grade atteint par un user.
- `UserCommands` — commande / panier d'un user (avec `isDistributor`, `state`, `motif`, totaux).
- `CommandProducts` — ligne d'une commande (produit, quantité).
- `UserCommandPackPromo` — achats de packs promotionnels.
- `UserPackComposition` — composition concrète d'un pack (sélection de produits).
- `UserPaidBonus` — historique des bonus payés (binaire, indirect, etc.).
- `UserBonusSpecial` — bonus spéciaux attribués individuellement.
- `SearchUser` — DTO entity pour recherche.

Catalogue & abonnements

- `Product` — produit du catalogue, avec prix client/distributeur et valeur SV.
- `ProductClientPrice`, `ProductDistributorPrice`, `ProductSV`, `ProductCote` — historique des prix et SV des produits.
- `PrestationService`, `Service` — services facturables.

- **Membership** — palier d'adhésion (carte / pack distributeur). Possède un coefficient (**coefficient**) : **1** = adhérent simple sans droit binaire ; **>1** = distributeur.
- **MembershipCost**, **MembershipSV**, **MembershipBonusPourcentage** — paramètres financiers d'un membership (coût, SV généré, % bonus binaire).
- **MembershipProduct**, **MembershipProducts**, **CompositionMembershipProductName** — produits inclus dans un membership.
- **MembershipSubscription** — souscription effective d'un user à un membership pour une période (clé du calcul cyclique).
- **PackPromo**, **PromoPackProduct**, **PromoBonusSpecial** — packs et promotions.
- **TVCPack** — TVA / package particulier.

Cycles, bonus et grades

- **Cycle** — période de calcul (semaine ou mois). Champs : **startedAt**, **endedAt**, **closed**, **binarySaved**, **weekly**, **autoSave**.
- **FiltreCycle**, **FiltreProduct** — filtres de listing.
- **SponsoringBonus** — paramétrage du bonus de parrainage direct.
- **BonusSpecial**, **CollectionBonusSpecial** — bonus spéciaux et leur regroupement.
- **IndirectBonusMembership**, **IndirectBonusProduct**, **CommissionIndirectBonus** — paramètres et historique du bonus indirect (commissions sur niveaux supérieurs).
- **LevelBonusGenerationnel** — niveaux du bonus générationnel (pourcentage par génération).
- **Grade**, **GradeLevel**, **GradeBG**, **GradeSV**, **GradeMaintenance** — définition des grades (palier de promotion) et conditions de maintenance.
- **SummaryCommission**, **PurchaseSummary** — synthèses pour rapports / paiement.

Notifications & technique

- **Notification**, **UpdateCartProductNotification**.
- **SendSMSPayment** — entité utilisée pour confirmer un paiement Mobile Money par SMS.
- **ParameterConfig** — table clé/valeur de paramètres généraux (utilisée par **ParameterConfigRepository::valueParameter('sv', \$cycle)** pour récupérer la valeur du SV courant).
- **AnalyseFonctionnelleSystematique** — module annexe d'analyse fonctionnelle systématique.

3.2 L'entité User en détail (src/Entity/User.php)

Classe centrale du domaine. Implémente **UserInterface**, **PasswordAuthenticatedUserInterface**, **EntityInterface**, **EntityWithImageToUploadInterface**. Annotée **#[Gedmo\Tree(type: 'nested')]**.

Champs principaux (lignes 36-220) :

Champ	Type	Rôle
id	int auto	PK

Champ	Type	Rôle
username	string(255) unique	identifiant de connexion
email, fullname, cni, city, country, mobilePhone, dateOfBirth, gender, title, documentType, nextOfKin	string/date	profil
entryDate	datetime	date d'inscription (auto en Africa/Douala via #[ORM\PrePersist] setStatusAccount() ligne 480)
position	enum Left/Right	position binaire sous le parent
category	enum admin/user	catégorie applicative
state	string (def. Actif)	état (Actif / Suspendu / ...)
password	string	hash bcrypt (auto dans security.yaml)
roles	json	par défaut ROLE_JTWC_USER
activated, expired, deleted, served, toUpgrade	bool	drapeaux de cycle de vie
dateActivation	datetime	date d'activation effective
sponsor	ManyToOne(User)	parrain (créateur logique de la lignée)
upline	ManyToOne(User)	upline binaire (parent immédiat dans l'arbre)
parent #[Gedmo\TreeParent])	ManyToOne(User)	parent dans l'arbre nested
children	OneToMany(User)	enfants directs (ordonnés par lvl/upline/position)
lft, rgt, lvl, root	int / User	colonnes nested set Gedmo
membership	ManyToOne(Membership) NOT NULL	adhésion courante
nextMembership	ManyToOne(Membership)	adhésion en attente d'upgrade
grade	string	label rapide du grade
userGrade	ManyToOne(Grade)	grade courant
codeDistributor	string	code distributeur unique (généré par GenerateUserDistributorCode)
imageFile / imageName	upload	photo de profil (Vich)
token	string	token applicatif (reset password / confirmations)
isConcernedByPromo	bool	éligibilité aux promos en cours
createdBy	ManyToOne(User)	créateur (admin ayant ajouté le user)

Champ	Type	Rôle
updatedAt	datetime	renseigné via #[ORM\PreUpdate] userUpdatedAt() (ligne 772)

Validations notables (groupes Symfony Validator) : `registration`, `quick_registration`, `update_profile`, `change_username`. Une contrainte custom `JTWCAssert\UplinePosition` (validateur dans `src/Validator/Constraints/`) empêche un placement binaire incohérent.

3.3 Diagramme textuel des relations clés



3.4 Migrations

47 migrations dans `migrations/` couvrant 2019-2021. Le schéma est aujourd'hui stable et la base de référence est rechargée depuis `docs/base de donnees/jessica_bd.sql` (cf. mémoire projet, mars 2026). Toute modification de schéma doit suivre la procédure : modification d'entité -> `make:migration` -> `commit` -> déploiement.

4. Logique métier MLM

4.1 Placement dans l'arbre binaire

L'arbre est implémenté avec **Gedmo Tree (nested set)** sur l'entité **User** (annotations `#[Gedmo\Tree(type: 'nested')]`, `TreeLeft`, `TreeRight`, `TreeLevel`, `TreeRoot`, `TreeParent`).

- Chaque user a au plus 2 enfants directs : un en position **Left** et un en position **Right** (`getPosition()`).
- Le **parent** (`TreeParent`) est le nœud immédiatement supérieur dans l'arbre binaire (= **upline**).
- Le **sponsor** est distinct : c'est le parrain commercial (celui qui a recommandé le filleul). Sponsor != upline si l'admin/distributeur place le filleul plus profond pour équilibrer.
- Une contrainte custom `App\Validator\Constraints\UplinePosition` (groupe **registration**) bloque l'inscription si la position choisie sous l'upline est déjà occupée.
- Un `Voter App\Voter\AddMemberVoter::ADD_MEMBER` (`src/Voter/AddMemberVoter.php`) interdit à un membre dont le **Membership** a `coefficient == 1` d'ajouter de nouveaux distributeurs.

Le **User** a aussi des helpers :

- `getDirectChildren(User, Cycle)` (via `UserRepository`) : enfants directs avec activité dans un cycle.
- `getUserNetwork(lft, rgt, Cycle)` : récupère tous les descendants (sous-arbre nested set) actifs dans le cycle.

4.2 Cycles, points et SV

Toute la mécanique de bonus est cyclique. Un **Cycle** correspond généralement à un mois (`weekly = false`) ou éventuellement à une semaine. Chaque cycle a `startedAt`, `endedAt`, `closed`, `binarySaved`, `autoSave`.

Le **SV** (Sales Volume / Volume d'affaires) est l'unité commune :

- Chaque produit déclare un **SV** (entité `ProductSV`) — c'est le PV générique du système.
- Chaque **Membership** déclare aussi un SV (`MembershipSV`).
- La valeur monétaire d'un SV à un instant donné est lue via `ParameterConfigRepository::valueParameter('sv', $cycle)` (ligne 140 de `BonusBinary.php`).
- Le service `ExtractSVFromCommands` (`src/Services/ExtractSVFromCommands.php`) extrait le SV à partir d'une liste de `UserCommands` (méthodes `getSVFromCommands`, `getSVAchatPersonnel`, `sommeSVAchatPersonnel`, `getSVAchatPack`).
- `GetVolumeSVGenerateByCycle` calcule le volume SV produit par un user/sous-réseau pour un cycle donné.

4.3 Bonus binaire (`BonusBinary`)

Cœur du système, dans `src/Services/BonusBinary.php` (~656 lignes). Algorithme `computeUserBonusGroup(User, Cycle, ParameterConfig $sv)` (lignes 268-418) :

- 1 Si l'utilisateur n'a pas d'enfants (`rgt - lft == 1`) ou si son `Membership` a `coefficient == 1`, retourne un récap nul.
 - 2 Récupère les **enfants directs** dans le cycle (`getDirectChildren`), puis pour chacun récupère son sous-réseau actif (`getNetworkOfMember -> UserRepository::getUserNetwork`). Ces utilisateurs sont rangés par côté `left / right`.
 - 3 Pour chaque côté, calcule le **SV d'achats personnels** de tous les membres (`getSVPersonalPurchase`). Stocké dans `results['al']` (achats jambe gauche) et `results['ar']` (achats jambe droite).
 - 4 Pour chaque côté, calcule le **SV de parrainage** (souscriptions à des memberships ou pack) :
 - Si `cycle->autoSave`, utilise `getSumSVMembershipSubscription`.
 - Sinon, somme `getAllSubscriptionOfCycle` puis `getSvGroupeNetwork`.
 - Stocké dans `results['pl'] / results['pr']`.
- 1 **Totaux par jambe** : `tl = al + pl, tr = ar + pr`.
 - 2 **Carry-over** : récupère via `UserMonthCarryOverRepository::getCarryOver` le report du cycle précédent (mensuel ou hebdo selon `cycle->weekly`).
 - 3 La jambe avec carry-over devient la **jambe forte** (`bm = côté + co_existant`). On calcule :
 - `binaire = min(jambe_forte, jambe_faible)` (le bonus est sur la jambe **faible**, le surplus passe en nouveau carry-over `n_co`).
 - `co_pos` indique le côté du nouveau carry-over.
- 1 Conversion en gain :
 - `sv_gain = (binaire * Membership.membershipBonusBinairePourcent) / 100`
 - `gain = sv_gain * ParameterConfig.sv` (valeur monétaire du SV à ce cycle)

Quand un cycle est clôturé (`closed && binarySaved`), `handleSavedCarryOver()` (ligne 425) relit simplement les valeurs persistées dans `UserMonthCarryOver`, garantissant l'idempotence des rapports historiques.

`getRecapBonusBinaire()` (ligne 102) génère le récap global de tous les networkers (`getAllActivatedNetworkers`) pour un cycle, et marque `paid` si tous les `UserPaidBonus` correspondants existent.

4.4 Bonus de parrainage direct

- Paramétré par `SponsoringBonus` (entité + repository).
- Service `PaidSponsoringBonus` (`src/Services/PaidSponsoringBonus.php`) marque comme payé un bonus de parrainage versé à l'upline lors d'une nouvelle souscription `Membership`.
- `GetRecapBonusSponsoringAndPersonalPurchase` (`src/Services/GetRecapBonusSponsoringAndPersonalPurchase.php`) calcule pour l'utilisateur courant le récap parrainage + achats personnels d'un cycle.

- L'événement `App\Event\ReferralBonusEvent` est dispatché par le `PaymentController` après activation d'une souscription pour déclencher le calcul (cf. injection `EventDispatcherInterface`).

4.5 Bonus indirect (commissions de niveau)

- Règles dans `IndirectBonusMembership` (commissions sur souscriptions) et `IndirectBonusProduct` (commissions sur achats produits).
- Service `IndirectBonusService` (`src/Services/IndirectBonusService.php`) parcourt l'upline pour répartir un pourcentage à chaque niveau supérieur.
- Persisté dans `CommissionIndirectBonus`.
- `GetUplineKnowingSponsor` (`src/Services/GetUplineKnowingSponsor.php`) résout la chaîne d'uplines à partir d'un user.

4.6 Bonus générationnel

- Niveaux configurés dans `LevelBonusGenerationnel` (génération N -> pourcentage).
- Service `GenerationalBonus` (`src/Services/GenerationalBonus.php`) explore les "génération" successives de filleuls (notion de profondeur basée sur le sponsor, pas la position binaire) et applique les pourcentages.

4.7 Bonus spéciaux et promo

- `BonusSpecial`, `PromoBonusSpecial`, `CollectionBonusSpecial` : configurables via `BonusSpecialController` et `PromoBonusSpecialController`.
- `GetEligiblePromoBonusForUser` détermine si un user remplit les conditions d'une promo.
- `UpdateUserSpecialBonusService` met à jour `UserBonusSpecial`.

4.8 Grades / rangs et promotion

- `Grade` représente un rang (ex. Bronze, Argent, Or). Lié à :
- `GradeLevel` : niveau exigé,
- `GradeBG` : Bonus Group (volume groupe exigé),
- `GradeSV` : SV requis,
- `GradeMaintenance` : conditions pour conserver le grade au cycle suivant.
- Service `App\Services\UserGrade` (`src/Services/UserGrade.php`) évalue à chaque clôture de cycle si l'utilisateur monte de grade et historise via `UserGrade` (entité).
- Le grade courant est stocké rapidement dans `User.grade` (string) + `User.userGrade` (FK).

4.9 Clôture de cycle

- `CloseCycle` (`src/Services/CloseCycle.php`) + `CloseCycleController` (`/admin/close/cycle/{id}`) :

- Calcule les bonus binaires de chaque networker, persiste les `UserMonthCarryOver` et `UserBinaryCycle`,
- Crée les `UserPaidBonus` correspondants,
- Met à jour les grades (`UserGrade` service),
- Marque le cycle `closed = true` et `binarySaved = true`.

4.10 Activité réseau et carry-over

- `MonthlyTurnOver`, `PersonnalNetworkActivity`, `ComputeBinaryTurnOverTrait` (utilisé par `BonusBinary`) : agrégats mensuels du chiffre d'affaires généré.
- `TurnOverController` expose les rapports correspondants aux admins.

5. Module produits & commandes

5.1 Catalogue

- Géré côté admin par `ProductController` (CRUD via `ProductHandler`), avec sous-controllers pour les prix : `ProductClientPriceController`, `ProductDistributorPriceController`, `ProductSVController`, `ProductCoteController`.
- Les produits ont des prix séparés client / distributeur. Les distributeurs voient les prix distributeur dans leur espace.

5.2 Workflow d'une commande

- 1 Le distributeur (ou un client via interface admin) ajoute des produits au panier (`UserCommands` en état "panier").
 - 2 `UserCommands::isDistributor` détermine le mode tarification (`getTotalDistributorPrice` ou `getTotalClientPrice`).
 - 3 Le panier non finalisé expire après N jours et est purgé par la commande CLI `jtwc:remove-expired-carts` (`src/Command/RemoveExpiredCartsCommand.php`).
 - 4 Lors du checkout, l'utilisateur saisit une `AddressUser` (téléphone Mobile Money inclus).
 - 5 La commande passe en `PurchaseSummary` (DTO entity, `src/Entity/PurchaseSummary.php`) :
 - `montant`, `motif`, `provider`, `opérateur`, `success`, `notifyPage`, `fail`, `transaction`, `addressUser`, `otpCode`.
-
- 1 L'utilisateur est redirigé vers le flux Dohone via `PaymentController::payOrder` (route `/ {provider} / {opérateur} / {id} / confirm / order / payment`, name `confirm_order_payment`, `PaymentController.php` ligne 107).
 - 2 Dohone notifie l'application sur l'URL `notifyPage` (`PUBLIC_ACCESS` dans `security.yaml`). En succès, l'événement `MembershipSubscriptionActivatedEvent` ou `ReferralBonusEvent` est dispatché et le `UserCommands` passe à l'état payé/livré.

5.3 Memberships

- `MembershipController`, `MembershipSubscriptionController` gèrent l'achat / l'upgrade.
- Une `MembershipSubscription` valide pour le cycle courant rend l'utilisateur "actif" et "networker" (éligible aux bonus).

6. Module paiements (Dohone)

6.1 Pattern Strategy

- `App\Services\Payment\PayInInterface` définit les méthodes :
`purchaseSummary(PurchaseSummary)`, `payIn($rcs = null)`, `setTelephone(int)`, `getProvider()`.
- `App\Services\Payment\PayInContext` (Strategy/Context) sélectionne l'implémentation par provider (actuellement `dohone`).
- `App\Services\Payment\PayInWithDohone` est l'unique implémentation. Elle injecte :
- `string $apiKeyDohone` (%env(API_KEY_DOHONE)% ou paramètre),
- `string $urlApiDohonePayIn` (URL API Dohone),
- `RouterInterface` (pour générer success/notify/fail URL absolus),
- `HttpClientInterface` (Symfony HttpClient).

6.2 Flux d'un paiement

`PayInWithDohone::payIn()` (lignes 80-124) :

- 1 Premier appel (`$rcs == null`) :
 - Construit la query : `cmd=start`, `rDvs=XAF`, `rH=apiKey`, `source=JessicaTWC`, `rMt=montant`, `rT=téléphone`, `rN=nom`, `motif`, `rMo=opérateur`, `rOTP=code`, `rI=transaction`, `endPage`, `notifyPage`, `cancelPage`.
 - GET sur l'URL Dohone, retourne le contenu (souvent un identifiant de session ou un message d'erreur `KO start` :).
- 1 Si Dohone exige une confirmation par OTP/SMS, l'app appelle `sendCfrmSMSCmd($rcs, $telephone)` qui envoie `cmd=cfrmsms`, `rCS=$rcs`, `rT=$telephone`.
- 2 Dohone notifie ensuite asynchroniquement l'URL `notifyPage` -> route admin de notification (`/admin/order-product/notify`, `/admin/order-membership-subscription/notify`, etc., toutes `PUBLIC_ACCESS` dans `security.yaml` lignes 36-44).
- 3 Le controller de notification (`PaymentController` + `CartNotificationController`) valide la transaction, crée `UserPaidBonus` si nécessaire, marque `MembershipSubscription` activée, dispatch les events bonus.

6.3 Remboursements / wallet

À ce stade, le code ne contient **pas** de portefeuille (Wallet) explicite. Les bonus sont consignés dans `UserPaidBonus` et payés "manuellement" (l'admin marque le bonus comme payé via `UserPaidBonusController`). Aucune sortie automatisée Dohone (PayOut) n'est implémentée — uniquement le flux PayIn.

7. Utilisateurs & sécurité

7.1 Configuration (`config/packages/security.yaml`)

- **Provider** : entité `App\Entity\User`, propriété `username`.
- **Hasher** : `auto` (bcrypt/argon2 selon disponibilité PHP).
- **Firewall main** :
 - pattern `^/`, lazy,
 - `custom_authenticator`: `App\Security\LoginFormAuthenticator`,
 - `user_checker`: `App\Security\UserChecker`,
 - `logout` via route `jwtc_app_logout`,
 - `remember_me` 7 jours via `DoctrineTokenProvider`.
- **Hiérarchie de rôles** :
 - `ROLE_JTWC_ADMIN` -> `ROLE_JTWC_USER_SECRET` -> `ROLE_JTWC_USER`.
- **Access control** :
 - `PUBLIC` : `/login`, et toutes les URLs de notification/success/fail Dohone (lignes 36-44),
 - tout le reste exige `ROLE_JTWC_USER`,
 - les controllers admin se protègent eux-mêmes via `#[IsGranted('ROLE_JTWC_ADMIN')]`.

7.2 LoginFormAuthenticator (`src/Security/LoginFormAuthenticator.php`)

- Lit `_username`, `_password`, `_csrf_token` (token id `jwtc_signing`),
- Cherche le user par `username`, lance `CustomUserMessageAuthenticationException('Username could not be found.')` sinon,
- Sur succès :
 - Si `state == 'Actif'` -> redirige vers `dashboard`,
 - Sinon -> redirige vers `new_user_update` (force la mise à jour du profil avant utilisation).

7.3 UserChecker

Vérifie probablement `deleted`, `expired`, `state` avant d'autoriser l'authentification (entité a tous les drapeaux nécessaires).

7.4 Voter

`AddMemberVoter` (`src/Voter/AddMemberVoter.php`) — attribut `add_member` : autorise un user à parrainer/ajouter un nouveau membre uniquement si son `Membership.coefficient != 1` (i.e. pas un simple adhérent).

7.5 Validators custom

`src/Validator/` contient `JTWCAssert\UplinePosition` qui s'assure qu'une position binaire (Left/Right) sous l'upline n'est pas déjà occupée (groupe `registration`).

8. Backoffice admin (controllers `Back/WebController/`)

Tous les controllers ci-dessous sont préfixés par `/admin` automatiquement. Liste exhaustive (40 controllers) :

Controller	Domaine
<code>DashboardController</code>	tableau de bord admin
<code>UserController</code>	gestion des distributeurs (liste, ajout depuis sponsor connu, génération codes distributeurs)
<code>UserCommandsController</code>	commandes / paniers
<code>UserGradeController</code>	grades attribués aux users
<code>UserPaidBonusController</code>	suivi des bonus payés
<code>BonusController</code>	récap bonus binaire / parrainage
<code>IndirectBonusController /</code> <code>IndirectBonusProductController /</code> <code>IndirectBonusMembershipController</code>	commissions indirectes
<code>LevelBonusGenerationnelController</code>	bonus générationnel
<code>SponsoringBonusController</code>	bonus de parrainage
<code>MembershipBonusPourcentageController</code>	% bonus binaire par palier
<code>PromoBonusSpecialController</code>	promotions et bonus spéciaux
<code>CycleController / CloseCycleController</code>	gestion / clôture des cycles
<code>ParameterConfigController</code>	paramètres généraux (valeur SV...)
<code>GradeController / GradeBGController /</code> <code>GradeSVController / GradeLevelController /</code> <code>GradeMaintenanceController</code>	définition des grades et conditions

Controller	Domaine
MembershipController / MembershipCostController / MembershipSVController / MembershipProductController / MembershipSubscriptionController / CompositionMembershipProductNameController	catalogue d'adhésions et leur composition
ProductController / ProductClientPriceController / ProductDistributorPriceController / ProductSVController / ProductCoteController	catalogue produits et historique de prix
PackPromoController / TVCPackController	packs promo et TVC
ServiceController / PrestationServiceController	services
PaymentController	paiement Dohone (voir §6)
CartNotificationController	notifications de paniers / paiements
TurnOverController	rapports de chiffre d'affaires / activité
AnalyseFonctionnelleSystematiqueController	module analyse fonctionnelle

Les controllers s'appuient quasi systématiquement sur leur Handler associé dans `src/Services/ModelHandlers/` pour exposer une interface CRUD homogène (liste paginée KnpPaginator + form Twig + redirection flash).

Exemple : UserController

```
#[IsGranted('ROLE_JTWC_ADMIN')]
#[Route('/users/all', name: 'user_list_all', options: ['expose' => true], methods: ['GET'])]
public function index(Request $request): Response { ... }

#[Route('/users', name: 'user_list', methods: ['GET'])] // accessible aux distributeurs : leur réseau
public function list(GetDownlinesGenealogyView $graph): Response { ... }

#[Route('/users/new', name: 'user_new_from_known_sponsor', methods: ['GET', 'POST'])]
public function addMember(Request $request): Response { ... }
```

(UserController.php lignes 74-120)

9. Espace distributeur

Bien que les controllers `Front/WebController/` et `Front/RestController/` soient **vides** dans le code actuel, l'espace distributeur partage en réalité le préfixe `/admin` : un distributeur (rôle `ROLE_JTWC_USER`) accède aux mêmes routes que l'admin mais filtrées par `#[IsGranted('ROLE_JTWC_ADMIN')]` ou par la logique métier (chaque service charge `tokenStorage->getToken()->getUser()` pour limiter aux données du user courant).

Templates dédiés :

- `templates/dashboard.html.twig`, `templates/user_dashboard.html.twig` — page d'accueil distributeur,
- `templates/admin_dashboard.html.twig` — page d'accueil admin,
- `templates/back/webcontroller/user/network.html.twig` — vue généalogique du réseau (utilisée par `UserController::list`),
- `templates/back/webcontroller/bonus/view_bonus_binaire.html.twig` — vue bonus binaire personnel,
- `templates/back/webcontroller/bonus/view_bonus_binaire_recap_cycle.html.twig` — récap admin par cycle.

Fonctionnalités distributeur principales :

- Voir son réseau / arbre binaire (généalogie via `GetDownlinesGenealogyView`),
- Ajouter un nouveau filleul (sous réserve `AddMemberVoter`),
- Consulter ses bonus (binaire, parrainage, indirect, générationnel) par cycle,
- Acheter des produits / souscrire à un nouveau membership,
- Consulter ses commandes et leur état,
- Mettre à jour son profil.

10. Routes principales (extrait)

Route name	URL effective	Controller	Rôle
<code>jtwc_app_login</code>	<code>/login</code>	<code>SecurityController</code>	PUBLIC
<code>jtwc_app_logout</code>	<code>/logout</code>	(firewall)	PUBLIC
<code>dashboard</code>	<code>/admin/dashboard</code> (ou similaire)	<code>DashboardController</code>	ROLE_JT WC_USER
<code>user_list_all</code>	<code>/admin/users/all</code>	<code>UserController::index</code>	ROLE_JT WC_ADMIN
<code>user_list</code>	<code>/admin/users</code>	<code>UserController::list</code>	ROLE_JT WC_USER
<code>user_new_from_known_sponsor</code>	<code>/admin/users/new</code>	<code>UserController::addMember</code>	ROLE_JT WC_USER + Voter <code>add_member</code>
<code>users_generate_code</code>	<code>/admin/users/generate-code</code>	<code>UserController::generateCode</code>	ROLE_JT WC_ADMIN
<code>cycle_list</code>	<code>/admin/cycles</code>	<code>CycleController</code>	ROLE_JT WC_ADMIN

Route name	URL effective	Controller	Rôle
cycle_new	/admin/cycles/create	CycleController	ROLE_JT WC_ADMIN
close_cycle	/admin/close/cycle/{id}	CloseCycleController	ROLE_JT WC_ADMIN
confirm_order_payment	/admin/{provider}/{opérateur}/{id}/confirm/order/payment	PaymentController::payOrder	ROLE_JT WC_USER
command_payment_notify	/admin/order-product/notify	PaymentController	PUBLIC (callback Dohone)
payment_product_success	/admin/payment-product/success	PaymentController	PUBLIC
payment_product_fail	/admin/payment-product/fail	PaymentController	PUBLIC
payment_subscription_success	/admin/payment-subscription/success	PaymentController	PUBLIC
payment_subscription_fail	/admin/payment-subscription/fail	PaymentController	PUBLIC

(Liste complète : ~200 routes — voir `php bin/console debug:router` ou les annotations `#[Route]` dans `src/Controller/Back/WebController/`.)

11. Commandes CLI personnalisées

Une seule commande Symfony custom :

`jtwc:remove-expired-carts`

Fichier : `src/Command/RemoveExpiredCartsCommand.php`.

- Argument : `days` (int, défaut 2).
- Supprime tous les `UserCommands` (paniers) non modifiés depuis plus de `days` jours, en streaming par lots avec `flush()` puis `clear()` pour limiter la mémoire.
- Repository concerné : `UserCommandsRepository::findCartsNotModifiedSince(DateTime)`.
- À exécuter via cron :

```
php bin/console jtwc:remove-expired-carts 2
```

12. Configuration & déploiement (référence rapide)

Local (dev)

- PHP 8.2 via XAMPP : `/c/xampp/php/php.exe`.
- Lancement serveur :

```
/c/xampp/php/php.exe -S localhost:8080 -t public public/router.php
```
- BD locale : MariaDB XAMPP, base `db_jessica_mlm` (root, pas de mot de passe).
- Cache Redis désactivé en dev (`config/packages/dev/cache.yaml` -> `cache.adapter.filesystem`).
- Sessions filesystem en dev (PdoSessionHandler incompatible avec le serveur PHP intégré).
- Login dev : `tiomelajorel@gmail.com` / `jorel5168`.

Production

- Hôte : `83.228.193.57` (alias SSH `jessica`).
- Path : `/home/ubuntu/jessica_mlm`.
- Docker Compose 3 conteneurs : `jessica_web` (Apache + PHP), `jessica_db` (MariaDB), `jessica_redis`.
- Fichier d'env : `.env.docker` (secrets prod, hors git).
- Déploiement :

```
docker compose --env-file .env.docker build --no-cache web
docker compose --env-file .env.docker up -d
```
- Reverse proxy Nginx + Let's Encrypt sur `https://jessica-mlm.duckdns.org`.
- CI/CD GitHub Actions sur push vers `main`.
- Backups DB quotidiens (`/home/ubuntu/db_backups/`) + Google Drive via rclone (script `backup_jessica_db.sh`, cron 22h).

Spécificités d'architecture Docker

- L'entrypoint Docker recrée `.env.local` à partir des variables d'environnement (Apache ne propage pas l'env du container vers PHP).
- Doctrine cache via `type: pool` (le `DoctrineProvider` a été supprimé en Symfony 6).
- Migration auto roles CSV -> JSON au premier démarrage (marker `var/.migration_v6_done`).
- Table sessions : colonne `sess_lifetime` en `INT` (et non `MEDIUMINT`) pour éviter l'overflow.

13. Glossaire MLM

Terme	Signification dans Jessica MLM
MLM	Multi-Level Marketing — vente directe en réseau avec rémunération multi-niveaux.
Distributeur	Membre actif possédant un Membership de coefficient > 1, capable de parrainer et d'accumuler des bonus.
Adhérent simple	User dont le Membership a coefficient == 1 (pas de droit binaire, pas de parrainage — bloqué par AddMemberVoter).
Sponsor / Parrain	User qui a recommandé un nouveau filleul (User.sponsor). N'est pas forcément l'upline binaire.
Upline	Chaîne ascendante des parrains/parents. User.upline = parent binaire immédiat ; getUplineKnowingSponsor remonte la chaîne.
Downline	Sous-réseau d'un user (tous ses descendants dans l'arbre nested set).
Arbre binaire	Structure d'arbre où chaque user a au plus 2 enfants directs : un en Left , un en Right . Implémenté avec Gedmo Tree (nested set : lft , rgt , lvl , root).
Leg gauche / Leg droite	Les deux jambes binaires d'un user. Le bonus binaire récompense la jambe faible .
Position	Left ou Right : place d'un user sous son upline (User.position).
PV / SV	Point Value / Sales Volume — unité de comptage du chiffre d'affaires servant au calcul des bonus. Chaque produit/membership porte un SV.
Cycle	Période de calcul (mensuelle ou hebdo) — entité Cycle . Un cycle est ouvert puis clôturé (closed , binarySaved).
Carry-over (CO)	Reliquat de SV de la jambe forte non binarisé qui se reporte au cycle suivant — entité UserMonthCarryOver .
BM (Bigger Member side)	"Big Member" — total de la jambe forte une fois ajouté l'ancien carry-over (variable bm dans BonusBinary).
TL / TR	Total jambe gauche / droite (tl = al + pl , tr = ar + pr).
AL / AR	Achats SV jambe gauche / droite (achats personnels du sous-réseau).
PL / PR	SV de parrainage (souscriptions Membership) jambe gauche / droite.

Terme	Signification dans Jessica MLM
Bonus binaire	Calculé chaque cycle : <code>min(jambe_gauche, jambe_droite) * %coeff_membership</code> puis converti en monnaie via <code>ParameterConfig.sv</code> .
Bonus de parrainage	Bonus direct versé au sponsor lors d'une nouvelle souscription <code>Membership</code> .
Bonus indirect	Commissions versées aux uplines au-delà du sponsor direct, paramétrées par <code>IndirectBonusMembership</code> / <code>IndirectBonusProduct</code> .
Bonus générationnel	Bonus payé sur N "générations" (profondeur de parrainage), par % défini dans <code>LevelBonusGenerationnel</code> .
Bonus spécial / Promo	Bonus exceptionnel attribué selon des règles configurables (<code>BonusSpecial</code> , <code>PromoBonusSpecial</code>).
Grade / Rang	Palier hiérarchique d'un distributeur (Bronze, Argent, Or...). Conditions définies par <code>GradeLevel</code> , <code>GradeSV</code> , <code>GradeBG</code> ; maintenance par <code>GradeMaintenance</code> .
Membership	Adhésion / pack distributeur acheté(e) par un user. Définit le coefficient binaire et le % de bonus.
Membership Subscription	Acte de souscription d'un user à un membership pour une période donnée.
Networker	User actif éligible aux bonus (<code>getAllActivatedNetworkers</code>).
Code distributeur	Identifiant commercial unique d'un distributeur, généré par <code>GenerateUserDistributorCode</code> .
Dohone	Agrégateur de paiement Mobile Money (Orange Money, MTN MoMo) intégré pour tous les paiements en XAF.
JTWC	Jessica The Winning Company — préfixe utilisé pour les rôles, commandes CLI et services internes.

Fin du document.